

MinZ Weekly: The Everything Sprint

253 commits, 8 frontends, 3 backends — March 11-18, 2026

Alice V. & Claude

March 18, 2026

Contents

1	The Everything Sprint	3
1.1	By The Numbers	3
2	Eight Frontends, One Pipeline	4
2.1	ABAP on Z80	4
2.2	ObjC with Dynamic Dispatch	5
2.3	Frontend Comparison	5
3	The LIR Backend Revolution	6
3.1	Why LIR?	6
3.2	Architecture: ISLE + WFC + PBQP	6
3.3	Results: 948/948 = 100%	6
3.4	Z80-Specific Innovations	7
3.5	LIR vs PBQP Comparison	7
4	FAT12/16 Filesystem	8
4.1	Operations	8
4.2	Verification: 5-Channel Pipeline	8
4.3	MIR2 Code Quality: Nanz vs C89	8
5	TUI Framework: Three Levels	10
5.1	Level 1: Flat API	10
5.2	Level 2: OOP (UFCS)	10
5.3	Level 3: Compile-Time Metafunction	10
5.4	Platform Support	10
5.5	Mini Norton Commander	11
6	Compile-Time Platform Detection	12
7	Native Backend Fixes	13
7.1	mir2qbe: ARM64 Pointer Store/Load	13
7.2	mir2c: Three Fixes	13
8	SDCC Comparison	14
9	Key Bug Fixes	15
10	Toolchain Updates	16

11 What's Next

17

Chapter 1

The Everything Sprint

253 commits. 553 files changed. +136,709 lines. Three parallel AI sessions. One week. This is what happened.

1.1 By The Numbers

Metric	Value
Commits (master)	253
Files changed	553
Lines added	136,709
New frontends	+1 (ABAP, now 8 total)
New backends	+1 (LIR/WFC, experimental → default)
New stdlib modules	4 (tui/widget, tui/render, tui/screen, fs/fat12)
New tools	mzn (native compiler via QBE/C99)
Bug fixes	27
Reports written	21 (#073 → #096)

Chapter 2

Eight Frontends, One Pipeline

All eight languages compile through a single $\text{HIR} \rightarrow \text{MIR2} \rightarrow \text{Z80}$ pipeline:

```
Nanz (.nanz)
C89/ObjC (.c)
PL/M-80 (.plm)
Lanz (.lanz)      → HIR  → MIR2  → Z80 / QBE / C99 / 6502
Lizp (.lizp)
Pascal (.pas)
ABAP (.abap)
HIR (.hir)
```

A function `double(x) = x + x` written in any of the eight languages produces the same Z80: `ADD A, A / RET`.

2.1 ABAP on Z80

The week's biggest frontend addition. ABAP — Advanced Business Application Programming — the language of SAP enterprise systems, now compiling to Z80 machine code.

```
REPORT zhello.
DATA lv_msg TYPE string VALUE 'Hello from ABAP on Z80!'.
WRITE lv_msg.
```

This compiles to **38 bytes** of Z80 for ZX Spectrum via RST \$10.

The parser is `@abaplint/core` (TypeScript by Lars Hvam Petersen) embedded as a Wasm blob — 14MB, no Node.js needed. Pipeline:

```
@abaplint/core → esbuild → Javy/QuickJS → .wasm → go:embed → wazero
```

Result: `go build` produces a single binary that parses ABAP with zero external dependencies.

Features working: `DATA`, `WRITE`, `IF/ELSEIF/ELSE`, `WHILE`, `DO/TIMES`, `CASE/WHEN`, `FORM/PERFORM`, `CLASS/INTERFACE/METHOD`, `PARAMETERS` selection screen, `SQLite` integration.

2.2 ObjC with Dynamic Dispatch

ObjC got `@protocol` vtables — dynamic dispatch via function pointer tables:

```
@protocol Effect
-(int)render:(int)t;
@end
```

```
@interface Plasma <Effect> { int scale; int speed; }
-(int)render:(int)t;
@end
```

Static dispatch: `[plasma render:0] → direct CALL Plasma_render.` Dynamic dispatch: `[(id<Effect>)obj render:0] → vtable lookup + indirect call.`

Three demoscene effects (plasma, diamond, XOR fractal) compile to both native (QBE/C99) and Z80.

2.3 Frontend Comparison

Frontend	Files	Asserts	Highlight
Nanz	95+	87+	Tetris, NC, FAT12, TUI, metafunctions
C89/ObjC	16	350+	FatFS R0.16, plasma renderer
PL/M-80	26	100%	Intel 8080 Tools corpus
ABAP	13	13/13 MZV	Wasm parser, selection screen
Pascal	6	—	Function pointers
Lanz	16 tests	—	S-expression interchange
Lizp	24 tests	—	Lisp-like syntax

Chapter 3

The LIR Backend Revolution

3.1 Why LIR?

The existing PBQP register allocator works well for simple functions but struggles with multi-block programs. Complex control flow causes register pressure spills, redundant moves, and missed optimization opportunities.

LIR (Low-level IR) takes a fundamentally different approach: **constraint propagation** instead of graph coloring.

3.2 Architecture: ISLE + WFC + PBQP

Three systems cooperating:

ISLE (Instruction Selection Language Engine) — pattern-matching rewrite rules for instruction selection:

```
(or (shl (load8 (add ?base (const 1))) (const 8))
    (load8 ?base))
→ (load16_le ?base)
```

This fuses two 8-bit loads into a single 16-bit load: 8 ops → 2 ops.

WFC (Wave Function Collapse) — constraint-based register allocation. Each virtual register starts with all possible physical registers as options. Constraints propagate: if an instruction requires A, the operand's domain collapses to {A}, which forces other live registers out of A.

PBQP acts as global strategist — providing hints from inter-procedural analysis that guide WFC's local decisions.

3.3 Results: 948/948 = 100%

Corpus	Functions (×3 machines)	Pass Rate
C89	720	100.0%
Nanz	162	100.0%

Corpus	Functions ($\times 3$ machines)	Pass Rate
Lizp	57	100.0%
Lanz	9	100.0%
Total	948	100.0%

3.4 Z80-Specific Innovations

IXH/IXL call-safe spill — the undocumented IX half-registers survive CALL instructions (callee doesn't touch them). LIR uses them as free spill slots: 8T vs 11T for PUSH/POP.

Tail call optimization — CALL fn; RET $\rightarrow$ JP fn. Saves 17 T-states and 1 byte per tail call.

Save-before-overwrite — Z80's destructive ALU operations (ADD overwrites A) require saving live values before they're clobbered. LIR inserts minimal save moves based on liveness analysis.

3.5 LIR vs PBQP Comparison

Example	PBQP	LIR	Winner
sum_array	16	13	LIR -18%
arena_allocator	723	456	LIR -36%
function_pointers	21	19	LIR -9%
ix_load_store	18	33	PBQP
four_pointers	15	27	PBQP
filter_map_chain	37	57	PBQP

LIR wins on large/call-heavy functions. PBQP wins on Z80-specific tricks (IX indexing, LUT access, conditional returns, DJNZ fusion). Five targeted improvements will close the gap.

Chapter 4

FAT12/16 Filesystem

A complete read-write FAT filesystem library in idiomatic Nanz (`stdlib/fs/fat12.minz`, 541 LOC).

4.1 Operations

Mount, find, read, create, delete, overwrite, cluster chain traversal. FAT12 and FAT16 auto-detection at mount time.

4.2 Verification: 5-Channel Pipeline

1. Nanz writes FAT image $\rightarrow$ Nanz VM reads back
2. Fresh Nanz VM reads (no state carryover)
3. gcc-compiled FatFS R0.16 reads (gold standard)
4. C89 MIR2 VM reads
5. Raw byte inspection

All channels agree — bit-for-bit identical.

4.3 MIR2 Code Quality: Nanz vs C89

Same algorithms, different frontends. MIR2 instruction counts:

Function	Nanz	C89	Delta	Notes
ld_word	9	9	TIE	
st_word	10	8	+2	Nanz: mask vs C89: trunc
read_fat12	15	16	-1	C89: extra move for pointer
sfn_checksum	18	19	-1	C integer promotion overhead
classify_fat12	21	21	TIE	
clst2sect	9	9	TIE	
chain_length	36	35	+1	break vs flag pattern

6 TIE, 2 Nanz wins, 2 C89 wins. The two frontends produce essentially identical code quality through the same backend.

Chapter 5

TUI Framework: Three Levels

A universal terminal UI framework that runs on CP/M, ZX Spectrum, and MZV.

5.1 Level 1: Flat API

```
sel_register_str(c"Name", c"World")
sel_show()
```

5.2 Level 2: OOP (UFCS)

```
var scr: Screen
scr.init(c"Material Report")
scr.add_field(c"Material", c"Steel")
scr.add_button(c"Execute")
scr.show()
```

5.3 Level 3: Compile-Time Metafunction

```
@screen("Material Report") {
  field "Material"
  field "Plant"
  button "Execute"
}
```

The `@screen` metafunction runs on MIR2 VM at compile time, iterates the block AST, and emits Nanz source via `emit()`. Lisp-style macros with normal syntax, written in the language itself.

5.4 Platform Support

Platform	Method	Binary Size
CP/M	VT100 via BDOS	735 bytes
ZX Spectrum	RST \$10 ROM codes	603 bytes

Platform	Method	Binary Size
MZV	ANSI host functions	—

5.5 Mini Norton Commander

474 lines of Nanz. Mounts FAT12/16 disk images, displays directory listing with cursor navigation, F3 file viewer, info panel.

```
./mzv --disk image.img examples/nanz/nc.nanz
```

Chapter 6

Compile-Time Platform Detection

New `@target()` intrinsic — returns a u8 constant at compile time:

```
if @target() == 0 { // CP/M
    asm z80 (in ch) { LD E, A / LD C, 2 / CALL 5 }
}
if @target() == 1 { // ZX Spectrum
    asm z80 (in ch) { RST 0x10 }
}
```

The constant folds through MIR2 optimizer. Dead branches eliminated by DCE. Same source, different targets, zero runtime cost.

Chapter 7

Native Backend Fixes

7.1 mir2qbe: ARM64 Pointer Store/Load

ObjC vtable pointers crashed on Apple Silicon — `storew` truncated 64-bit pointers to 32-bit. Fix: `storel/loadl` when the register carries a pointer value.

7.2 mir2c: Three Fixes

1. **Alloca register declaration** — `OpAlloca` dst wasn't declared
2. **Symbol sanitization** — `@` and `.` in names (`@mir2.str.0`) → `_mir2_str_0` for valid C identifiers
3. **`void main()` → `int main()`** — clang requires it

Both backends now compile ObjC plasma to native. QBE: 52KB, C99: 35KB.

Chapter 8

SDCC Comparison

Identical C source compiled through MinZ C89 and SDCC 4.2.0:

Function	MinZ	SDCC	Delta
twice	2B	3B	−1B
add	2B	3B	−1B
max	12B	12B	TIE
abs_diff	9B	11B	−2B
sum_to	21B	25B	−4B
clamp8	10B	30B	−20B
minmax	19B	61B	−42B
smaller	0B	34B	−34B
TOTAL	81B	179B	−55%

MinZ generates 55% less code than SDCC on these benchmarks. Key advantages: multi-return ABI, conditional returns (`RET Z/C`), EQU degenerate functions.

Chapter 9

Key Bug Fixes

Bug	Impact	Fix
ARM64 adrp w0	ObjC vtables crash	storel/loadl for pointers
mir2c alloca	Struct allocations fail	Emit variable declaration
DJNZ elimJrToRet	Exit RET eaten	Guard peephole
FAT cache offset	Multi-cluster corrupt	Sector-relative offsets
ABAP string DATA	WRITE prints nothing	Intern string + assign addr
Import cycle lanz isle	LIR branch won't build	Extract test helpers
String pool merge	@screen fails	remapStringRef s

Chapter 10

Toolchain Updates

- **VSCode v0.8.0** — all 9 frontends, native compile in context menu
- **MZV** — TUI hosts, disk I/O, `--zx/--disk` flags
- **MZE** — CP/M file I/O, Zork I runs
- **MZN** — canvas C runtime, auto-harness, all 8 frontends
- **MZX** — RZX input replay, headless mode

Chapter 11

What's Next

1. LIR conditional returns (RET cc) — biggest impact: −20-50%
2. IX-indexed patterns in ISLE — struct access optimization
3. NC dual-panel + directory navigation
4. `@target()` in TUI stdlib — single source for all platforms
5. Metafunction expansion — `@test`, `@derive`, `@sql`

MinZ: Modern programming abstractions with zero-cost performance on vintage Z80 hardware.